

RESEARCH PAPER · EVIDENCE THROUGH 2026-06-22

RAG 前沿 2026 从检索流水线 到可学习的知识系统

一份从经典 RAG、Advanced RAG 到 Agentic、可学习搜索与多模态检索的完整技术读物，并复核国内外大厂的公开落地路径。

Winston Research Notes

V1.0 · 2026-06-22

证据截止 2026-06-22，面向 RAG 工程师、研究者与技术决策者

| 目录

01	执行摘要：RAG 正在从固定管线变成检索决策系统	03
02	经典 RAG：外部记忆如何进入生成过程	05
03	工程基线：数据、切片、召回、重排与引用	07
04	Advanced RAG：在流水线的每个薄弱点增加判断	09
05	长上下文、GraphRAG 与多粒度知识组织	11
06	Frontier：Agentic RAG 与可学习搜索策略	13
07	多模态 RAG：从 OCR 文本回到原始页面	15
08	评测与安全：把答案质量拆回可诊断的组件	16
09	国内外大厂实践：平台能力正在如何收敛	18
10	生产蓝图：先做最小基线，再为困难查询购买复杂度	19
11	研究议程、术语表与延伸阅读	21
12	参考文献：30 个核心来源与证据等级	23

执行摘要

检索增强生成(Retrieval-Augmented Generation, RAG)的关键变化,是检索决策权正在从固定配置移向模型策略。2020 年的经典 RAG 把一次检索结果交给生成器;2023 至 2024 年的 Advanced RAG 开始判断是否检索、检索结果是否可信、需要哪种粒度;2025 至 2026 年的前沿工作进一步训练模型规划查询、调用搜索、阅读证据并决定何时停止。[S01, S05-S08, S11-S14]

七个结论先行

1. **RAG 是外部记忆与生成模型之间的接口,不等于向量检索。**经典论文把参数化记忆与可更新的非参数化记忆结合起来,向量索引只是其中一种实现。[S01]
2. **检索质量仍是第一约束。**DPR、CoBERTv2 等工作提升了语义召回与细粒度匹配;2026 年已部署客服研究发现,只要相关文档进入前三条,超过 92% 的案例能生成准确、完整的回答。[S02, S03, S21]
3. **没有一种 Advanced RAG 技巧可以无条件迁移。**HyDE 在其零样本检索基准中有效,但在一项生产客服研究里 Recall@10 从 66.4% 降至 62.8%;BM25 在 T2-RAGBench 的文本表格任务中价值,却会拖累另一组强稠密检索组合。[S04, S18, S21]
4. **长上下文与 RAG 应由路由器组合。**2024 年对比研究中,资源充足时长上下文平均表现更强;RAG 更便宜,输入远超窗口时仍占优。Self-Route 据此把性能与成本变成逐查询决策。[S10]
5. **Frontier RAG 的核心是学习搜索策略。**Search-R1、ReSearch 与 DeepResearcher 把搜索视为可交互环境,用强化学习优化「何时搜、搜什么、搜几次」;A-RAG 则把关键词、语义片段与全文阅读暴露成分层工具。[S11-S14]
6. **多模态文档不应默认先压扁成文本。**CoIPali 与 VisRAG 直接检索页面图像,保留表格、布局和图示;VisRAG 在其设置中报告相对 TextRAG 约 20%-40% 的端到端提升。[S15, S16]
7. **生产 RAG 是受治理的数据系统。**权限过滤、来源血缘、更新时效、拒答、成本和攻击面与 Recall 同等重要。PoisonedRAG 表明,少量恶意文本即可操纵回答,外部知识库不能视为可信边界。[S20, S22-S27]

本文的判断框架

本文把 RAG 拆成三个平面。知识平面负责数据进入、解析、切片、索引、权限和更新;检索平面负责查询理解、召回、融合、重排与上下文压缩;控制平面负责路由、迭代、工具调用、停止条件与成本预算。经典 RAG 主要建设前两个平面,Frontier RAG 的新增价值集中在第三个平面。

CORE THESIS

RAG 正在把每一次信息获取变成可观测、可评测、可预算的决策。检索策略越自由，系统越需要权限、来源与停止条件约束。

证据边界

本文共使用 30 个核心来源，证据截止北京时间 2026-06-22。正式论文与预印本分开处理；云平台文档证明公开能力，不证明实际效果；厂商 benchmark 仅按厂商自报呈现。不同来源出现冲突时，正文保留条件差异，不把冲突抹平成一条万能经验。完整映射见 `sources.md` 与 `data/source-map.tsv`。



图 1: RAG 的控制权从静态配置移动到模型策略，工程约束同时从相关性扩展到轨迹与治理。

经典 RAG：外部记忆如何进入生成过程

经典 RAG 解决的是知识存储位置问题。语言模型把知识压进参数后，更新慢、来源难追踪；RAG 把一部分知识放到可检索的外部索引，在推理时选取证据，再让生成器基于证据作答。[S01]

原始定义比今天的产品口径更严格

Lewis 等人的 2020 年论文把文档视为潜变量，检索器返回文档分布，生成器对候选文档条件下的输出概率做边缘化。RAG-Sequence 在整段输出中使用同一文档，RAG-Token 允许不同 token 依赖不同文档。今天工程中常见的「先检索再拼 Prompt」保留了外部记忆思想，却通常不再端到端训练检索器与生成器。[S01]

这个区分很重要。研究模型 RAG 关心联合概率、端到端训练与任务指标；应用系统 RAG 关心异构数据、权限、更新、延迟、引用与可运维性。两者共享名字，但失败路径并不相同。工程系统即使不训练任何参数，也能通过索引与上下文编排获得价值。

一个最小可用闭环

离线：文档 → 解析 → 切片 → 元数据 → 向量 / 倒排索引
在线：问题 → 查询规范化 → 召回 top-k → 重排 → 上下文组装
生成：系统约束 + 问题 + 证据 → 回答 + 引用 + 拒答状态
反馈：查询、命中、引用、延迟、人工纠错 → 评测集与索引更新

最小闭环里没有 Agent、知识图谱或多模型路由。它仍然可以上线，但必须满足四条约束：文档有稳定 ID 和版本；每个切片保留父文档与权限元数据；答案能回指证据；未检索到充分证据时允许拒答。缺少任意一条，系统都难以调试或治理。

检索器的三种表达能力

检索方式	匹配信号	擅长	常见失误
稀疏检索	词项、词频、倒排索引	专有名词、编号、精确短语	同义表达和语义改写
单向量稠密检索	查询与文档整体向量相似度	语义近似、自然语言问题	细粒度实体、数字与否定条件
多向量 / late interaction	token 级向量交互	细粒度相关性与跨域检索	索引体积、服务复杂度

DPR 用双塔编码器展示了稠密检索在开放域 QA 中的可行性，在其基准上相对强 BM25 系统把 top-20 检索准确率提高 9 至 19 个百分点。[S02] CoBERTv2 保留 token 级 late interaction，并用残差压缩把索引空间相对原 CoBERT 降低 6 至 10 倍。[S03] 这些结果说明检索表达力可以购买，但没有证明稠密检索在每种企业语料上都胜过词法匹配。

RAG 的四类收益

1. **更新**: 知识库替换或新增文档，无需重新训练生成模型。
2. **长尾**: 模型参数未覆盖的专业知识可以在推理时提供。
3. **来源**: 检索结果可作为引用候选，为核验建立入口。
4. **控制**: 企业可以通过数据范围、权限过滤与版本策略界定回答边界。

RAG 降低了「知识不在模型参数里」这一类错误，却增加了检索遗漏、错误证据、上下文冲突、权限泄漏与知识库投毒。它改变错误结构，不消除错误。

工程基线：数据、切片、召回、重排与引用

生产 RAG 的质量上限通常在生成前就已确定。解析丢掉表格，切片切断条件，权限元数据没有进入过滤器，后面的 reranker 和大模型只能更流畅地处理不完整证据。

数据进入索引前要保留什么

对象	最低字段	原因
文档	document_id、version、source_uri、updated_at	更新、去重、删除与来源追踪
切片	chunk_id、parent_id、section_path、offset	父子回溯与上下文重建
权限	tenant、principal、ACL、classification	检索时安全裁剪，而非生成后遮盖
质量	parser、language、content_type、confidence	识别 OCR、表格、图片等弱环节
时效	effective_from、expires_at、supersedes	处理制度版本与冲突来源

切片不是固定 token 数

切片要同时服务召回和阅读。过短切片容易命中，却丢掉定义、例外和表头；过长切片保留语境，却稀释相关信号并增加重排成本。更稳妥的做法是按标题、段落、表格和代码结构切分，并保留父子关系：子切片负责匹配，父单元负责回填上下文。

一项 2026 年已部署客服研究使用 15,848 份帮助文档与 310 个专家反馈查询，发现 2,048-token 切片在其数据上通常优于 512-token 切片，原因是较长单元保留了帮助文章的连贯性。[S21] 这个反例不能推导出新的默认长度，却说明只按通用博客经验设置 300 或 500 token，可能错过真实语料的结构。

召回与重排要分开看

第一阶段召回追求覆盖，第二阶段重排追求前列精度。候选集过小，reranker 无法找回遗漏文档；候选集过大，交叉编码器或 LLM 重排的成本迅速上升。生产指标应同时报告 Recall@k、nDCG 或 MRR、最终引用覆盖率与每次查询成本，不能只看一个端到端正确率。

已部署客服研究给出一个实用信号：相关文档出现在前三条时，超过 92% 的回答准确且完整；主要缺口是 Recall@50 到 Recall@10 的排序损失。[S21] 这类证据把优化目标从「换更大的生成模型」改成「把已召回证据推到可用位置」。

混合检索不是信仰

稀疏与稠密信号互补时，可以用 Reciprocal Rank Fusion 或学习融合提高覆盖。但相同做法在不同数据上会相反。T2-RAGBench 的 23,088 个文本表格问答样本中，Hybrid BM25 是最强检索方案；另一项客服研究则发现，在四个数据集上把 BM25 加到强稠密模型会降低整体表现。[S18, S21]

ENGINEERING RULE

先建立词法、稠密、混合三条可复现基线，再决定是否融合。任何没有在本域查询集上验证的「最佳实践」，都只是待检验的候选配置。

引用需要原子化

把文档 URL 放在回答末尾不等于引用。可核验引用需要把回答拆成原子主张，记录每个主张对应的证据片段、文档版本与位置；若证据只支持一半句子，引用就不完整。长答案应限制每段的独立主张数，避免一条引文被迫支撑多个事实。

Advanced RAG: 在流水线的每个薄弱点增加判断

Advanced RAG 是一组针对局部失败的干预方法，分别处理查询表达不足、检索噪声、长文层级、生成途中缺证据与错误上下文。选择方法前，应先定位失败发生在哪一层。

查询侧：改写、扩展与分解

HyDE 先让语言模型生成一篇假设文档，再用该文档向量去寻找真实文档，试图跨过短查询与长文档之间的表达差距。[S04] 它适合语义意图清楚但词汇不匹配的零样本检索，不适合把虚构细节当作事实，也未必适合精确数值问题。2026 年客服研究在 300 个查询子集上观察到 HyDE 的 Recall@10 从 66.4% 降至 62.8%，说明查询扩展可能把检索带离原问题。[S21]

多查询改写与问题分解解决的是复合意图。一条问题同时含时间、地区、指标和例外条件时，单向量容易把条件平均掉。拆成多个子查询并行召回可提高覆盖，但会放大调用次数、融合难度和权限检查面。只有当评测集确实包含复合查询时，分解才值得成为默认路径。

生成侧：主动检索、反思与校正

方法	触发机制	解决的问题	代价
FLARE	预测下一句，低置信 token 触发检索	长文生成途中出现新信息需求	临时生成、反复检索与重写 [S05]
Self-RAG	训练反思 token 判断检索与支持度	固定 top-k、无关证据、引用不一致	需要专门训练数据与解码逻辑 [S06]
CRAIG	轻量评估器判断检索质量	本地检索错误或不不确定	增加评估器与外部网页搜索依赖 [S07]

FLARE 把一次性检索改成生成期信息获取；Self-RAG 把「是否需要搜」与「证据是否支持」编码成模型可生成的反思 token；CRAIG 在检索后增加质量评估和纠错动作。[S05-S07] 三者共同完成了一次概念迁移：检索不再是固定前处理，而是生成过程中的条件动作。

知识侧：层级与全局问题

普通 chunk 检索擅长局部事实，不擅长回答「整个语料的主要主题是什么」。RAPTOR 递归聚类并摘要，建立从原始片段到高层概括的树；其论文中，RAPTOR 与 GPT-4 在 QuALITY 上报告 82.6% 准确率，相比此前 62.3% 的最佳结果提高约 20 个百分点。[S08] 这个结果来自特定长文 QA 设置，也伴随索引构建与摘要失真风险。

Microsoft GraphRAG 用实体关系图、社区检测和社区报告支持全局 query-focused summarization。[S09] 它适合需要跨文档主题、关系和群体结构的问题，不应替代精确事实检索。图谱构建使用 LLM 抽取时，实体合并错误会固化进索引，查询成本也高于平面向量检索。

常见误区

1. 先上 GraphRAG，再寻找需要全局图问题的业务。
2. 把查询改写后的文本当成事实，而不是召回工具。
3. 只评最终答案，不知道提升来自召回、重排还是生成器。
4. 用 LLM judge 替代所有人工标注，没有校准 judge 的偏差。
5. 增加 Agent loop 后，不设置最大步数、成本与证据充分度。

| 长上下文、GraphRAG 与多粒度知识组织

RAG 与长上下文的选择应落实到逐查询路由。数据规模、查询范围、时效、权限、延迟与证据密度共同决定哪条路径更合适。

2024 年对比研究给出的条件化答案

Li 等人在 9 个长上下文数据集上比较 RAG 与完整长上下文。三种当时的模型中，长上下文平均分别领先 RAG 7.6%、13.1% 和 3.6%；RAG 在输入显著超过模型窗口时仍有优势。[S10] 论文提出 Self-Route：先用检索上下文判断能否回答，不能回答的查询再升级到完整长上下文。

这项研究还揭示 RAG 失败的四类原因：多跳问题需要前一步答案才能继续检索；全局问题难以形成局部查询；长复杂查询难被单向量理解；隐式问题需要理解整段材料。[S10] 这些失败不是增加 top-k 就一定能解决，往往需要迭代检索、摘要层级或完整上下文。

三条路径的工程边界

路径	优先场景	优势	主要风险
经典 RAG	局部事实、语料很大、更新频繁	便宜、快、权限与引用易控制	召回遗漏、复杂问题一次检索不足
长上下文	单份材料可放入窗口、全局理解	保留整体语境、少建索引	成本、延迟、位置偏差、权限裁剪粗
Agentic RAG	多跳、跨源、需要交叉验证	按需探索、可组合多种工具	轨迹不稳定、调用放大、攻击面扩大

FIGURE 2 QUERY ROUTING

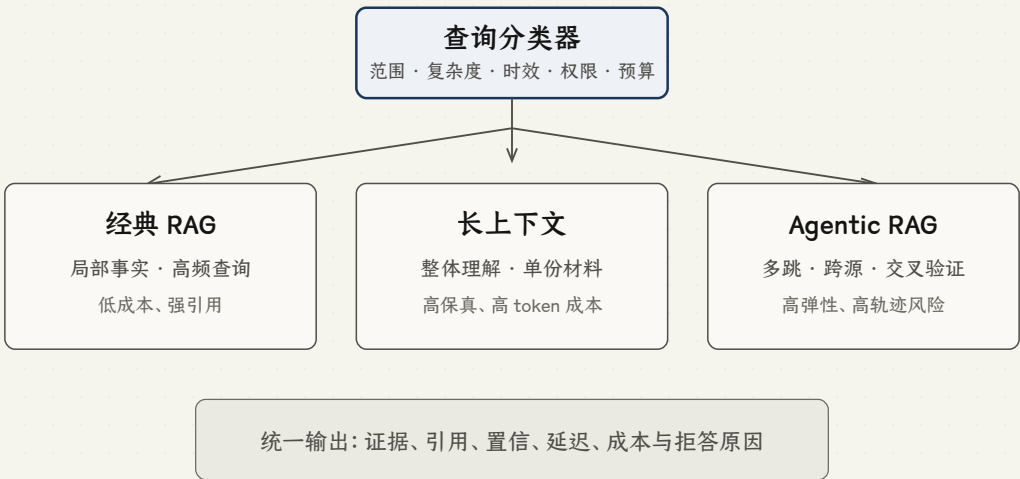


图 2: 路由器不只看上下文长度, 还要联合查询范围、权限和预算; 三条路径共享同一评测与治理出口。

GraphRAG、层级摘要与长上下文保留不同结构

RAPTOR 按语义层级组织摘要树, GraphRAG 按实体关系和社区组织全局报告, 长上下文直接保留原始顺序。三者分别优先保留层级、关系和原文, 损失函数不同。选择结构时要先问问题需要什么: 跨层摘要、关系路径, 还是原文细节。

| Frontier: Agentic RAG 与可学习搜索策略

Frontier RAG 把搜索从函数调用升级成序列决策。模型不只消费 top-k 结果，还要决定查询、工具、阅读粒度、下一步和停止时机。这个自由度可以提升复杂问题表现，也会把延迟、循环和错误传播带进系统。

从提示工程到策略学习

Search-R1 把搜索引擎作为强化学习环境，轨迹交替包含思考、搜索与检索结果；它用 retrieved token masking 避免把环境返回内容当作策略输出训练，并用结果奖励优化多轮搜索。COLM 2025 PDF 的摘要与结论在 7 个 QA 数据集上，对 Qwen2.5-7B 与 3B 报告相对 RAG 基线 24% 和 20% 的平均提升；同一版本的结果概述与 arXiv 页面却写成 41% 和 20%。本文采用较低的 24% 口径，并保留这一版本内冲突。[S11]

ReSearch 同样用强化学习让模型学会在推理中搜索，但强调不依赖人工标注的推理过程。[S12] DeepResearcher 把训练扩展到真实网页环境，WebDancer 则把浏览数据构造、轨迹采样、监督冷启动和强化学习组织成四阶段训练范式。[S13, S30] 这些工作共同指向一个变化：检索质量不仅取决于 encoder，还取决于长期动作策略。

A-RAG 把复杂度移到接口

A-RAG 的 2026 年预印本提供三个工具：keyword_search、semantic_search 与 chunk_read。模型先看关键词或语义摘要，再决定是否读取完整 chunk，避免把所有候选一次性塞进上下文。[S14] 这比预先写死 GraphRAG 或工作流更接近 Agent：方法的关键不再是更复杂的检索算法，而是更清晰的工具粒度与返回契约。

A-RAG 在 MuSiQue 的实验中，把最大步骤从 5 提到 20 时，GPT-5-mini 的表现约提高 8%；把 reasoning effort 从 minimal 提到 high 时，GPT-5-mini 与 GPT-5 约提高 25%。[S14] 这些是预印本设置下的结果，尚不能直接外推为生产收益。它们更可靠地说明了一个研究方向：RAG 开始具备 test-time scaling 轴。

FIGURE 3 AGENTIC RETRIEVAL LOOP

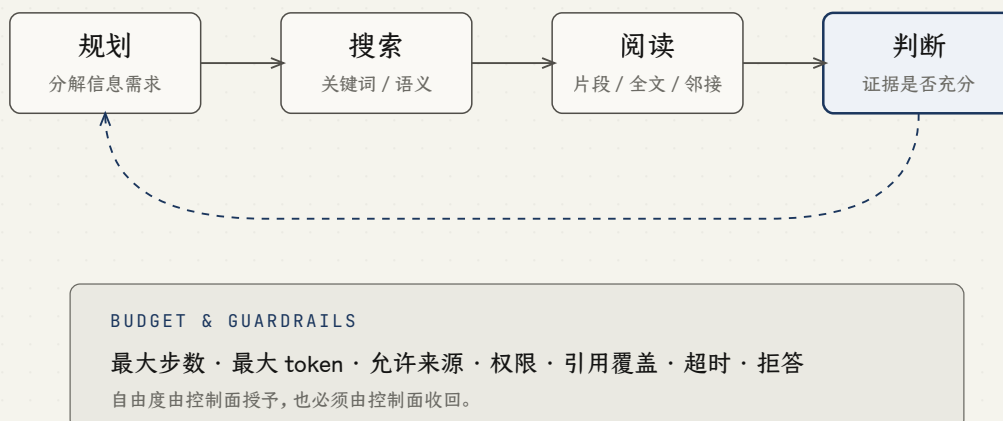


图 3: Agentic RAG 的循环必须与预算和安全约束并列设计；没有停止条件的自主检索只是不可控放大器。

Deep Research 是 RAG 的长时程产品形态

OpenAI 将 Deep Research 描述为经过端到端强化学习的网页研究 Agent，可在多步轨迹中搜索、阅读文本、图像和 PDF，并回溯与引用。[S28] Anthropic Research 强调跨网页与内部 Workspace 连续搜索，并提供可检查引用。[S29] 这些页面证明多轮检索已经进入公开产品，但没有公开足够细节让外部复现其全部架构。

Frontier 的未决问题

- 停止：什么时候证据已经足够，而不是继续搜索相似页面。
- 奖励：只奖励最终答案是否会学到投机搜索或过度调用。
- 轨迹：如何评估中间查询、来源选择、回溯和证据覆盖。
- 稳定：网页变化、搜索排序与工具失败会让结果难以复现。
- 治理：Agent 能访问更多源后，权限、投毒与提示注入风险同步增加。

| 多模态 RAG: 从 OCR 文本回到原始页面

真实知识不只存在于段落。财报表格、论文公式、产品手册布局和演示文稿图示在 OCR 之后经常失去关系。多模态 RAG 的前沿工作因此把「页面」重新定义为可检索对象。

ColPali: 页面图像作为多向量文档

ColPali 用视觉语言模型把整页图像编码为多向量表示，再用 late interaction 匹配文本查询与页面 token。它同时发布 ViDoRe 基准，覆盖不同领域、语言与视觉文档任务。[S15] 这条路线减少了 OCR、版面分析与文本拼接的脆弱级联，但需要保存页面图像表示，并承担视觉模型的索引和服务成本。

VisRAG: 检索与生成都保留页面

VisRAG 用 VLM 同时承担页面检索与答案生成，直接绕过传统 TextRAG 的解析阶段。论文在 6 个数据集上报告，使用不同生成器时相对 TextRAG 获得约 20% 至 40% 的端到端提升；其离线页面处理在 PlotQA 设置中为 121 ms，而文本基线的解析加编码为 312 ms，在线查询总时延相同为 54 ms。[S16]

这些数字来自指定硬件、模型与数据集，不能直接作为采购指标。更稳妥的结论是：当回答依赖版面、图表或视觉关系时，纯文本解析产生的信息损失可能大于 VLM 成本。文本密集、结构简单的语料仍可能更适合成熟的文本检索。

表格是检索与推理的双重难点

T2-RAGBench 由 23,088 组问题、上下文和答案构成，包含 7,318 份真实金融文档；专家确认 91.3% 的改写问题在脱离上下文后仍具有唯一语义。[S18] 它要求系统先从文本与表格语料中找到正确文档，再做数值推理，避免只在 oracle context 下测生成器。

文档形态	建议主索引	补充索引	验证重点
制度与帮助文章	结构化文本切片	关键词、元数据	版本、例外条款、父段落
财报与数据表	表格结构或页面视觉	文本说明、公司与期间过滤	表头、单位、期间、计算链
论文与技术手册	页面视觉 + 文本双索引	章节层级、公式说明	图文对应、跨页引用
图片与幻灯片	视觉多向量	OCR、标题、演讲者备注	布局、图例、页面顺序

平台实践正在吸收多模态解析

Amazon Bedrock Knowledge Bases 建议包含图表、表格或图片的 PDF 使用 Bedrock Data Automation 或基础模型解析，并允许把抽取出的视觉元素存入 S3，在检索结果或来源归因中返回。[S24] 阿里云百炼公开支持文档、表格、图片与多模态向量，腾讯知识引擎把公式、图片、表格和阅读顺序纳入文档解析能力。[S25, S26]

评测与安全：把答案质量拆回可诊断的组件

端到端正确率告诉你系统坏了，却不告诉你坏在哪里。RAG 评测必须区分知识库覆盖、召回、排序、上下文利用、答案忠实度、引用与运行成本；Agentic RAG 还要评估整条轨迹。

最小评测矩阵

层	核心问题	指标示例	需要的标注
语料	答案是否存在、是否最新、权限是否正确	coverage、freshness、ACL violation	文档版本与授权真值
召回	相关证据是否进入候选集	Recall@k	问题到相关文档
排序	相关证据是否足够靠前	MRR、nDCG@k	分级相关性
生成	答案是否正确且只用证据支持的内容	exact match、faithfulness、citation precision	答案或原子主张
系统	成本与延迟是否可接受	p50/p95、tokens、tool calls、cost	运行日志
轨迹	查询、工具与停止是否合理	step success、source diversity、loop rate	轨迹审核规则

RAGAS 的价值与边界

RAGAS 把自动评测分成忠实度、答案相关性与上下文相关性。其 WikiEval 实验中，与人工偏好的符合率分别为 0.95、0.78 和 0.70，高于直接让 GPT 打分或排序的基线。[S19] 上下文相关性最难，且长上下文会让评估模型更难选出真正必要的句子。

自动评测适合高频回归，不适合代替金标。每个业务至少保留一组人工审阅的困难查询，覆盖无答案、冲突来源、过期制度、多跳、表格、权限和恶意文档。LLM judge 更换模型或 Prompt 后，应重新校准与人工判断的一致性。

安全从知识库完整性开始

PoisonedRAG 把知识库识别为新的攻击面。攻击文本需要同时满足两个条件：容易被目标问题检索到，并能诱导生成器输出攻击者指定答案。论文在百万级干净文本的知识库中，为每个目标问题注入 5 条恶意文本，报告约 90% 的攻击成功率；在 NQ 黑盒设置中达到 97%。[S20]

这类攻击不能只靠 Prompt 防御。需要在数据进入、索引、检索和生成四处建立控制：来源 allowlist 与签名；文档变更审计；重复与异常相似度检测；可信度参与排序；对冲突证据显式标注；高风险答案要求多源一致或人工复核。权限必须在检索前过滤，因为生成后删除敏感文字无法撤回模型已经读取的内容。

上线门槛

1. 困难查询集有稳定版本，并记录来源与期望行为。
2. 可区分无文档、未召回、排序失败、生成误用与引用错误。
3. 每个回答能回放查询、候选、重排、最终上下文与模型版本。
4. 权限、删除和版本替换在索引与缓存中可验证生效。

5. 注入、投毒、冲突来源和超长输入经过对抗测试。
6. p95 延迟、单次成本和最大工具调用数有明确预算。

| 国内外大厂实践：平台能力正在如何收敛

公开产品没有收敛到同一套内部实现，却明显收敛到同一组产品能力：托管摄取、多格式解析、混合检索、重排、引用、权限、观测与 Agentic 查询规划。差异更多落在数据生态、可控粒度、成熟度和计费方式。

海外平台

平台	公开能力	可提炼的落地模式	边界
Microsoft Azure AI Search	LLM 查询分解、并行子查询、关键词/向量/混合搜索、语义重排、引用与活动日志	把检索层产品化为 Agent 的知识接口	部分能力 GA、部分仍为 preview；多查询增加 token 与延迟 [S22]
Google Vertex AI RAG Engine	摄取、转换、corpus 索引、检索与生成的托管框架	与 Cloud Storage、Drive 和区域治理整合	区域成熟度和安全控制支持不完全一致 [S23]
Amazon Bedrock Knowledge Bases	文本与多模态解析、图表抽取、S3 归档、来源归因、reranker	把文档理解与知识库摄取合并	高级解析按页或 token 产生额外成本 [S24]
OpenAI Deep Research	多步网页搜索、文件、图像与 PDF、Python 工具、引用	把 RAG 延伸为异步研究任务	架构细节与评测主要来自厂商描述 [S28]
Anthropic Research	连续网页搜索、Google Workspace、内部文档索引、inline citations	把私域工作上下文与开放网页统一检索	官方页面证明产品形态，不足以复现内部策略 [S29]

国内平台

平台	公开能力	可提炼的落地模式	边界
阿里云百炼	文档、表格与图片知识库，命中测试，多知识库召回，Rank 重排，RDS/OSS 接入	在应用与 workflows 中复用统一知识库能力	最多 5 个知识库等限制随产品更新，需按文档复核 [S25]
腾讯云	解析、切片、embedding、改写、重排原子能力；Agentic RAG 支持反思、策略切换和多轮检索	同时提供组件化数据平面和 Agent 控制平面	2026-05-28 的 Agentic RAG 页面属于平台公开能力，不是独立效果评测 [S26]
百度千帆 AppBuilder	RAG、Agent、工作流、AI 搜索、iRAG 与低代码/高代码工具链	围绕企业应用交付整合知识、工具和 UI	公开页面以产品能力为主，缺少可横向比较的检索实验 [S27]
阿里巴巴 WebDancer	浏览数据、轨迹采样、SFT 冷启动与 RL 的四阶段研究路线	把深度搜索能力沉淀为可训练 Agent	论文 benchmark 不等于云产品 SLA [S30]

行业收敛出的五条工程原则

1. 数据接入是一等产品。解析、更新、权限和版本比聊天界面更难替换。
2. 召回与生成保持解耦。检索响应包含证据、引用和执行信息，允许下游 Agent 自主消费。[S22]
3. 多查询并行而非无限串行。复合问题分解后并行召回，减少 Agent 链路的线性延迟。
4. 多模态解析进入默认能力。图表、表格和图片不再只靠 OCR 文本代表。[S24-S26]
5. 可观测性进入接口。查询计划、来源、活动日志与成本成为 RAG 响应的一部分，而不是排障时临时拼接。

生产蓝图：先做最小基线，再为困难查询购买复杂度

可上线的 RAG 路线不从 GraphRAG 或 Agent 开始。它从一组真实问题、可追踪语料和可诊断基线开始；复杂度只用于已经被评测确认的失败类型。

案例：企业售后知识助手

假设知识库包含 20,000 份产品手册、服务政策和维修公告，查询分成三类：60% 是单一事实，25% 同时包含型号、地区和购买时间，15% 需要跨手册、工单和网页公告交叉验证。设计目标是让每类查询走成本最低的可靠路径。

查询	首选路径	升级条件	输出约束
「X200 的电池保修多久？」	混合召回 + rerank	无有效版本或地区冲突	政策版本、适用地区、原文引用
「2024 年在上海购买的 X200，进水后能否保修？」	条件分解 + 元数据过滤 + 多查询	政策条款与维修公告冲突	逐条件判定，不满足时拒绝下结论
「过去一年 X200 进水故障的主要原因与处置建议？」	Agentic RAG / 层级摘要	证据少于 2 类来源或覆盖不足	区分统计、案例与厂商建议，列出样本边界

四个可独立上线的阶段

- 1. 基线：结构化解析、稳定 ID、权限过滤、词法与稠密检索对照、引用和拒答。先覆盖单一事实。
- 2. 高级检索：加入父子切片、融合与 reranker，只针对 Recall@k 或排序失败。阶段完成后仍是可用的经典 RAG。
- 3. 路由：按查询复杂度选择经典 RAG、长上下文或结构化工具。记录路由正确率与成本差异。
- 4. Agentic：只为多跳和交叉验证查询开放多轮搜索，设置来源 allowlist、最大步数、token、超时与引用覆盖门槛。

一套可执行的默认参数观

默认值应来自校准，不来自框架示例。初始候选可以设置多个 k 值并画出 Recall、nDCG、延迟与成本曲线；切片可以从结构边界与 512/1,024/2,048 token 三档起测；reranker 深度只覆盖 Recall 曲线仍增长的区间。每个新增模型或工具都必须证明它在困难查询子集上的边际收益。

失败路由表

观察到的失败	优先排查	可能修复	不要先做
答案文档不在 top-50	语料覆盖、解析、查询表达	修数据、混合召回、领域 embedding	换更大生成模型
在 top-50, 不在 top-5	排序与候选噪声	rerank、融合、元数据过滤	扩大上下文到全部候选
证据正确, 答案错误	Prompt、冲突证据、上下文利用	原子主张、引用约束、模型路由	继续增加召回量
多跳问题缺后续证据	问题分解与停止条件	Agentic loop、结构化工具	把所有查询改成 Agent
表格或图示问题失败	解析损失	表格结构索引、视觉 RAG	只调 chunk overlap
回答引用了过期制度	时效元数据与版本替换	effective date 过滤、冲突提示	让 LLM 自己判断新旧

上线验收

一条查询必须能够从答案回放到证据、候选、查询计划、语料版本、权限决策、模型版本和成本。无法回放的 RAG 系统即使平均分很高，也不具备稳定迭代条件。

| 研究议程、术语表与延伸阅读

2026 年之后，RAG 的主要增量不会只来自更强 embedding。更关键的问题是检索策略如何学习、外部知识如何保真、多模态记忆如何统一，以及系统如何在性能、成本与可信之间动态分配计算。

六条前沿研究方向

1. 可学习检索策略：从最终结果奖励走向过程约束，避免过度搜索、投机查询和来源单一。
2. Agent-friendly interfaces：比较关键词、语义、图遍历、SQL、页面阅读等工具粒度，形成稳定的返回契约。[S14]
3. 成本感知 test-time scaling：让步数、reasoning effort、检索深度与模型大小共同受预算控制。
4. 多模态统一记忆：文本、表格、页面、图像、音频和结构化数据共享检索与引用坐标。[S15-S18]
5. 可验证生成：从回答级 citation 走向原子主张、证据充分度、冲突与不确定性表达。
6. 安全与隐私检索：研究知识库投毒、提示注入、访问控制、来源信誉与私有检索的可证明边界。[S20]

研究判断

最可能稳定落地的方向是经典 RAG 与 Agentic RAG 共存：高频局部问题走低成本管线，困难问题由路由器升级到多查询或 Agent。平台文档已经体现这种双轨结构，Azure 同时保留 classic 与 agentic retrieval，腾讯也把原子 RAG 能力与 Agentic RAG 分层提供。[S22, S26]

最值得谨慎的方向是把开放网页搜索直接接入高权限 Agent。搜索范围扩大后，知识库投毒、提示注入、隐私泄漏和工具误用会形成级联风险。前沿性能必须与来源 allowlist、只读工具、权限继承、轨迹日志和人工升级机制同时评估。

术语表

RAG: Retrieval-Augmented Generation，在生成时引入外部检索证据。

Advanced RAG: 对查询、索引、召回、重排、上下文与生成加入适应性或校正机制的统称，不是单一算法。

Agentic RAG: 模型可自主规划并多轮调用检索工具，根据结果决定下一步与停止。

Hybrid Retrieval: 融合词法与向量等多类检索信号，效果依赖语料与融合策略。

reranker: 对第一阶段候选重新打分的模型，常见为 cross-encoder、late interaction 或 LLM。

Late Interaction: 查询与文档分别编码为多个 token 向量，在检索时做细粒度交互。

GraphRAG: 利用实体关系、社区或图遍历组织检索的 RAG 家族，适合关系和全局语料问题。

Multimodal RAG: 检索与利用文本之外的页面、图像、表格、音频等证据。

Test-time Scaling: 推理时通过更多步骤、搜索或计算提高表现，而不是只扩大训练规模。

Further Reading

起点	资料	为什么读
最佳起点	RAG 原始论文 [S01]	理解外部记忆与生成的原始定义
工程检索	已部署客服研究 [S21]	看到 BM25、HyDE、融合、重排与切片在真实语料上的条件化结果
Advanced	Self-RAG [S06]	理解按需检索与反思如何进入模型解码
Frontier	Search-R1 [S11]	理解搜索引擎如何进入强化学习环境
2026 前沿	A-RAG [S14]	理解 Agent-friendly 分层接口与 test-time scaling，注意其预印本身份

结论

经典 RAG 把知识从参数中解耦，Advanced RAG 把检索流程拆成可优化组件，本文所称 Frontier RAG 则把信息获取交给可学习策略。三者对应成本与复杂度不同的工具层，可以长期共存。成熟系统会保留简单路径，并只让困难查询进入更昂贵的搜索循环。

FINAL PRINCIPLE

先证明「答案存在并能被找回」，再优化「模型如何使用答案」；先让困难查询可识别，再让 Agent 获得更多自由。RAG 的可靠性来自边界清楚，不来自组件数量。

12 · REFERENCES

| 参考文献: 30 个核心来源与证据等级

文中编号与本表一一对应。A 表示正式会议或期刊论文, B 表示研究预印本, C 表示官方产品文档, D 表示厂商自报效果。平台文档只能证明公开能力, 不能替代独立效果评测。

[S01] A Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. [arXiv](#)

[S02] A Karpukhin et al. Dense Passage Retrieval for Open-Domain Question Answering. EMNLP 2020. [arXiv](#)

[S03] A Santhanam et al. ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. NAACL 2022. [arXiv](#)

[S04] A Gao et al. Precise Zero-Shot Dense Retrieval without Relevance Labels. ACL 2023. [arXiv](#)

[S05] A Jiang et al. Active Retrieval Augmented Generation. EMNLP 2023. [arXiv](#)

[S06] A Asai et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR 2024. [arXiv](#)

[S07] B Yan et al. Corrective Retrieval Augmented Generation. 2024 preprint. [arXiv](#)

[S08] A Sarthi et al. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. ICLR 2024. [arXiv](#)

[S09] B Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. Microsoft Research, 2024. [official page](#)

[S10] A Li et al. Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach. EMNLP 2024 Industry Track. [arXiv](#)

[S11] A Jin et al. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. COLM 2025. [arXiv](#)

[S12] B Chen et al. ReSearch: Learning to Reason with Search for LLMs via Reinforcement Learning. 2025 preprint. [arXiv](#)

[S13] B Zheng et al. DeepResearcher: Scaling Deep Research via Reinforcement Learning in Real-world Environments. 2025 preprint. [arXiv](#)

[S14] B A-RAG: Scaling Agentic Retrieval-Augmented Generation via Hierarchical Retrieval Interfaces. 2026 preprint. [arXiv](#)

[S15] A Faysse et al. ColPali: Efficient Document Retrieval with Vision Language Models. 2024. [arXiv](#)

[S16] A Yu et al. VisRAG: Vision-based Retrieval-augmented Generation on Multi-modality Documents. 2024. [arXiv](#)

[S17] B M2RAG: A Multimodal Retrieval-Augmented Generation Benchmark. 2025 preprint. [arXiv](#)

[S18] A T2-RAGBench: A Comprehensive Benchmark for Retrieval-Augmented Generation with Text and Tables. EACL 2026. [ACL Anthology](#)

[S19] A Es et al. RAGAS: Automated Evaluation of Retrieval Augmented Generation. EACL 2024. [arXiv](#)

[S20] A Zou et al. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. USENIX Security 2025. [arXiv](#)

[S21] A Retrieval Enhancements for RAG: Insights from a Deployed Customer Support Chatbot. EACL 2026 Industry Track. [ACL Anthology](#)

[S22] C Microsoft. Agentic Retrieval in Azure AI Search. Updated 2026-06-12. [official docs](#)

[S23] C Google Cloud. Vertex AI RAG Engine Overview. Updated 2026-06-18. [official docs](#)

[S24] C Amazon Web Services. Advanced Parsing Options for Amazon Bedrock Knowledge Bases. Checked 2026-06-22. [official docs](#)

[S25] C 阿里云. 百炼知识库. 复核于 2026-06-22. [官方文档](#)

[S26] C 腾讯云. Agentic RAG. Updated 2026-05-28. [官方文档](#)

[S27] C 百度智能云. 千帆 AppBuilder 知识库. 复核于 2026-06-22. [官方文档](#)

[S28] C/D OpenAI. Introducing Deep Research. Updated 2026-02-10. [official page](#)

[S29] C/D Anthropic. Research. 2025-04-15. [official page](#)

[S30] B Wu et al. WebDancer: Towards Autonomous Information Seeking Agency. 2025 preprint. [arXiv](#)